

Scalar Replacement of Objects

Malay Kedia(23b1034) and Vishwaajith N.K(23b1038)

April 19, 2026

1 What are we doing?

Scalar replacement

Scalar replacement is an optimization technique where we replace an object with its individual fields and also convert any function calls with that object as the receiver to static calls. This is done to reduce the number of heap accesses and allocations, which are expensive operations. But this optimization is a per object optimization which is only possible under very specific conditions. These conditions are

- there is no instance in which there is a local variable which may point to this object and some other object.
- there is no instance in which there is a global variable which may point to this object.
- there is no instance in which there is a field which may point to this object.
- there is no instance in which it is passed as an argument to a method which changes the state of the object.
- It does not escape the method in which it is created.
- It never has its identity used in any way, for example, it is never compared with another object using `==` or it is never used as a key in a hash map.

Rather than trying five or six different optimizations each in a shallow way, we chose to go deep on one. Most of our time was spent on soundness: making sure every allocation we scalar-replace is one that's safe to replace, and that the rewritten code is semantically equivalent to the original. We have tried to be complete in our implementation, and we have not left any corner cases unhandled. We have also decided to be conservative in our implementation, and would rather miss some opportunities for scalar replacement than to perform an unsafe optimization.

2 What analysis did we use?

The tool is a three-pass pipeline registered on Soot's whole-program `wjtp` pack.

Pass-1: per-method summaries: For every user method we run a forward dataflow that tracks, for each local, which of the enclosing method's parameters it may directly alias. From that we derive a 'MethodSummary': which parameters escape (return, throw, stored to static or to another object's field), which are used for identity (`'=='`, `'!='`, `'instanceof'`, `'synchronized'`), which are forwarded to callees whose parameters are themselves

bad, which fields of each parameter are read, and which are written. A caller checks the method summary to determine if it can scalar-replace an argument. Pass 1 runs in reverse topological order of the call-graph SCCs computed by Tarjan, with pessimistic handling of cycles: every method in a non-singleton SCC is marked fully bad and stays that way.

Pass-2: per-allocation replaceability: For each method we run a second forward dataflow, this time tagging each local with the set of allocation sites it may alias. When the alloc is used as the base of a field access we record the field in the allocation’s `fieldsUsed` set; when it’s passed to an invoke we consult the callee’s summary and either accept the call (good callee with empty modified-field set — we’ll specialize it later) or disqualify the alloc. Identity use, heap-store on the alloc’s ref, ambiguous aliasing (the local may refer to multiple allocations), and passing to a mutating callee are disqualifiers.

2.1 Pointer analysis: dimensions of precision

Both passes are may-alias forward dataflow analyses. Our analysis is Flow-sensitive, Path-insensitive and Context-insensitive. Beyond that, some other features of our analysis are:

- **Interprocedural via method summaries:** at a call site the forward dataflow queries the callee’s previously-computed `MethodSummary` to decide how the call affects the caller’s aliases.
- **Field-sensitivity:** partial. `readFields` and `modifiedFields` in `MethodSummary` track individual `SootFields`, so we know *which* fields a callee reads vs. writes on a given parameter. The abstract state itself, however, does not model the heap: we do not track what’s in *o.f* for any abstract object *o*. So Pass 1 is field-sensitive for reads and writes on parameters but heap-insensitive for objects we don’t control.
- **Allocation-site sensitive:** In Pass 2 every `new` statement is its own abstract object, so two `new X()` at different source lines in the same method are distinct allocations with independent verdicts and independent scalar expansions. Arrays are never tracked.
- **Object-insensitive:** we don’t refine summaries by dynamic type and we don’t do heap cloning. For virtual dispatch we take the pessimistic union over all CHA/SPARK targets: a call is specialisable only if *every* possible concrete callee is specialisable at that position.

3 How do we transform the code?

Pass-3: IR rewrite. For every surviving allocation we create one fresh local per accessed field, named `sr0<line>_<idx>_<field>`. Before the inlined constructor runs we splice in default-value writes to each of these scalars (zero/null/false) so reads that precede an explicit write behave like a freshly-allocated heap object.

The constructor chain (validated by Pass 2 to be well-behaved) is then inlined at the allocation site by `InitInliner`. For each level of the chain we clone every local with the unique name `sr_il<counter>_<origName>`. Identity statements for `@this` are dropped; `@parameterN` becomes a direct `local = actual` binding; a `this.f = v` store in the constructor becomes `sr_f = v`; a `super-<init>` call on `this` is replaced by a `NopStmt` and the recursion splices the super’s translated body in its place. Every dropped statement leaves a `NopStmt` placeholder. The `new X`; and the original `<init>` call are deleted and insertion is just before the `<init>` invoke.

After the alloc-site rewrite we walk the rest of the caller body: every field access whose base unambiguously aliases a scalar-replaced allocation (per Pass 2’s flow-sensitive `AllocState`) is redirected to the matching scalar local.

Non-`<init>` calls receiving the allocation become static invokes of a synthesised specialisation produced by `Specializer`. The specialisation is keyed on (callee, set of scalarised positions). Its name is `<origName>$scalar-<positions>` (e.g. `foo$scalar_0_1`). Each scalarised reference parameter in the signature expands into one primitive/ref parameter per field the callee reads, named `srP<pos>_<field>`; non-scalarised parameters pass through unchanged. We clone the body with `Body.importBodyContentsFrom`, rewrite `@this/@parameter` identity statements to match the widened signature, replace field reads on aliases of the scalarised parameter with reads of the corresponding `srP` local, and recursively specialise any nested invoke that forwards a scalarised parameter.

4 Partial scalar replacement: what we tried and why we dropped it

The obvious next step after “kill allocations that don’t escape” is “kill allocations that only escape on some paths”. If an object leaks into the heap only inside an if branch that’s rarely taken, we’d rather keep the scalars on the hot path and materialize a real object just before the escape.

Where we came unstuck was generalising the soundness conditions. The easy cases — single escape point on a branch, no post-escape field writes, simple constructors without static side effects — were clean. But as soon as we tried to combine them (multi-escape paths, control-flow joins where some predecessors had materialized and some hadn’t, constructors with `counter++` style side effects that would double-execute if we called `<init>` at materialization) we ran into cases where a particular design choice was either too strict (giving up on obviously-safe cases) or too loose (producing semantically wrong output on testcases we couldn’t enumerate in advance). We got to a point where each fix for one shape reintroduced a failure on another.

5 Results

We ship ten single-file Java testcases in the `tests` directory that cover a variety of scenarios. Wall-clock timings are under `-Xint`, averaged over five timed runs of three iterations each, with one warm-up run first to populate caches. Every row includes an automatic correctness check: the stdout of the baseline and the optimized run must match byte-for-byte before the timing is recorded.

5.1 Results on DaCapo benchmarks

With the core implementation working we tried applying it to the TamiFlex-prepared DaCapo 9.12 benchmarks. The analyzer ran end-to-end on four out of five benchmarks (batik ran out of heap in our per-local dataflow state at 6 GB). The transformation completed in roughly a minute per benchmark and produced valid class files that ran to completion with output identical to the baseline.

The result on timing was dead flat:

Zero net speedup. The reason is exactly what the analyzer itself tells us: each benchmark has a handful (3 to 8) of allocations that pass all our checks, and the rest (millions of them per run) are ruled out by the soundness conditions. Of the few we do replace, none

Test	Scenario	Allocs	Base (ms)	Opt (ms)	Speedup
1	one alloc and one field	1	736	184	75%
2	Multi-field with constructor arg	1	909	311	65%
3	Passed to good-callee helper	2	936	356	61%
4	Nested specialization (two levels)	2	1246	606	51%
5	Three-level super chain	1	1426	379	73%
6	Negative case: identity use	0	507	489	3%
7	Negative case: mutating callee	1	964	922	4%
8	Virtual dispatch devirtualized	1	1201	375	68%
9	Negative case: heap escape	1	140	141	0%
10	Cross / dot vector pipeline	4	3796	2103	44%

Table 1: Testcase wall-clock averages under `-Xint` (5 runs $\times$ 3 iterations; warm-up discarded). Positive tests show 44–75% speedup; negatives stay within noise of baseline, as intended.

Benchmark	Y verdicts	.class emitted	Base (ms)	Opt (ms)	
avroa	8	5915	621	629	+1.3%
batik	—	—	—	—	OOM (>6 GB)
fop	5	8936	22644	22613	~0%
luindex	4	5554	27739	27675	~0%
xalan	3	6025	40015	40804	-2%

Table 2: DaCapo 9.12 on `-Xint`, 5 timed runs per configuration, 3 iterations each. All runs produce baseline-identical output.

happen to sit on a hot loop. Avroa’s replaced allocations are in ELF-parsing startup code that runs once per invocation; xalan’s are in rarely-executed initialization.